Nand2Tetris

Project 3 — HDL Implementations

Name: Nimarta Kaur Thind

Roll no: 2501420013

Course: B.Tech CSE (Data Science)

Section: A

Semester: 3

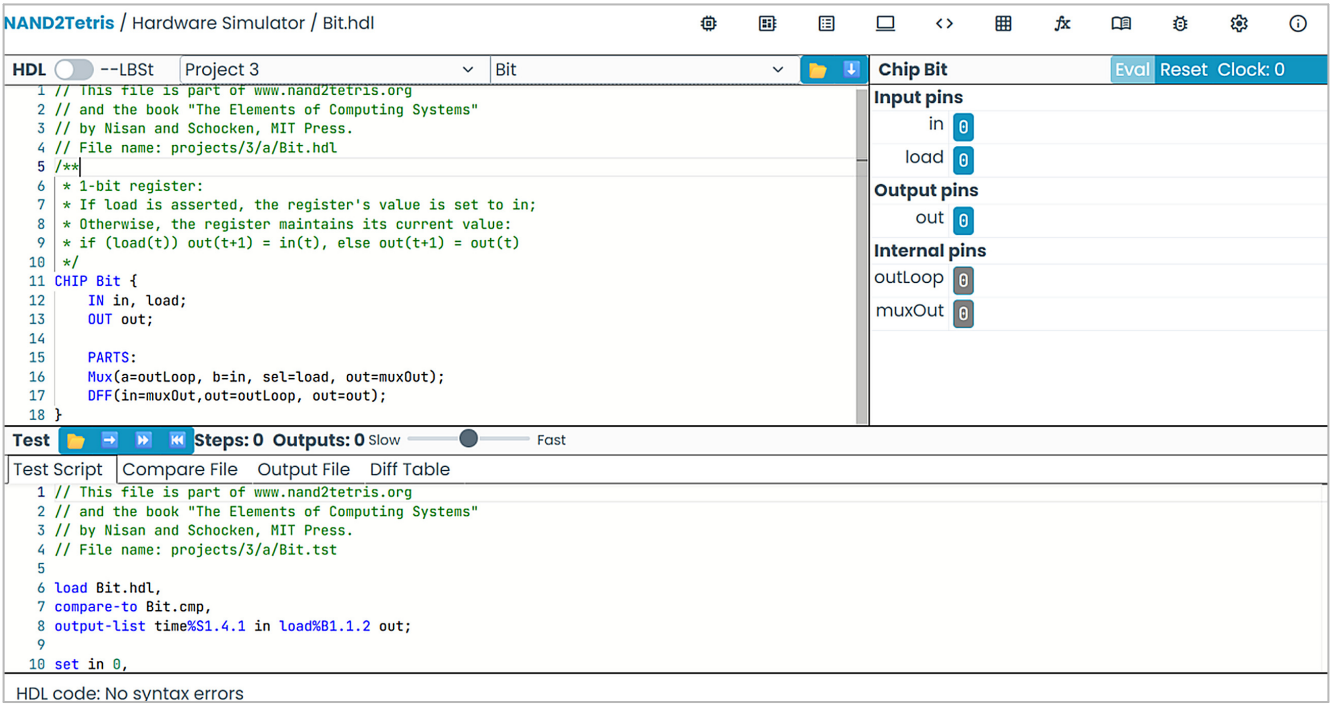
Project 3: Sequential Logic

Implementation of memory elements and sequential chips — from the basic data flip-flop up through the Hack computer's 16K-register RAM hierarchy and program counter — built on top of the Project 1 and Project 2 chip libraries, as specified in Chapter 3 of The Elements of Computing Systems.

Bit

```
CHIP Bit {
    IN in, load;
    OUT out;

    PARTS:
    Mux(a=outLoop, b=in, sel=load, out=muxOut);
    DFF(in=muxOut, out=outLoop, out=out);
}
```

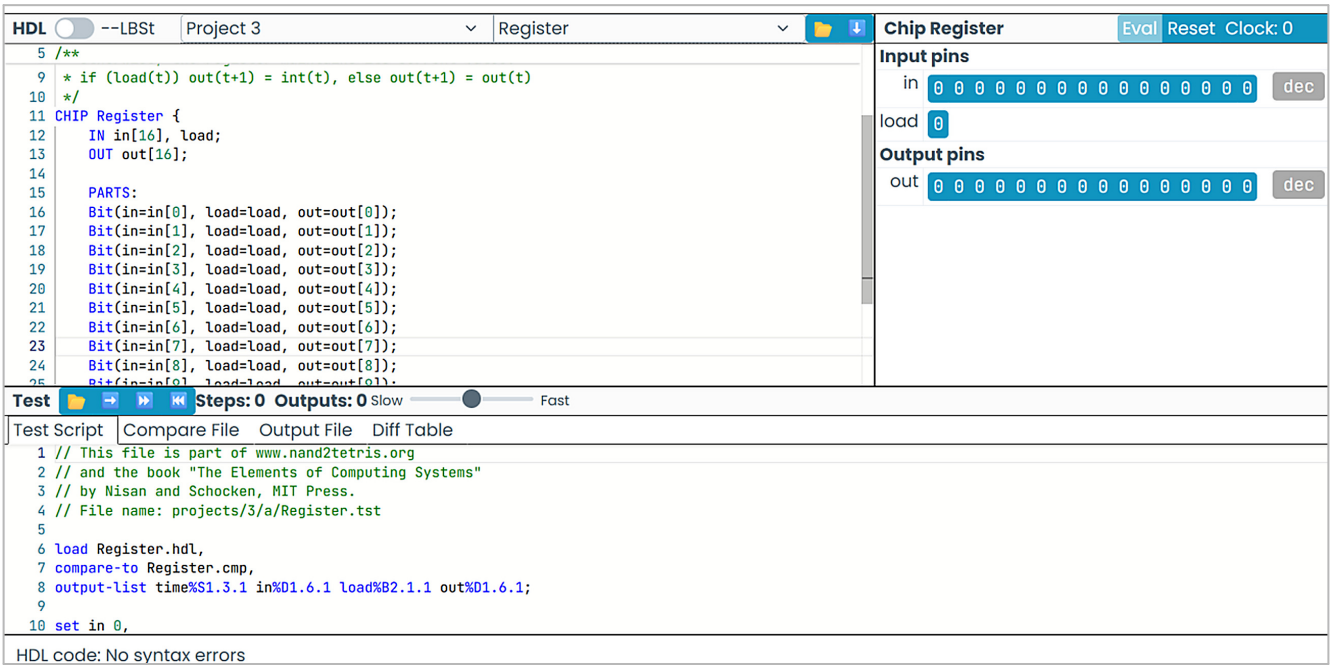


Simulator output: Bit

REGISTER

```
CHIP Register {
    IN in[16], load;
    OUT out[16];

    PARTS:
    Bit(in=in[0], load=load, out=out[0]);
    Bit(in=in[1], load=load, out=out[1]);
    Bit(in=in[2], load=load, out=out[2]);
    Bit(in=in[3], load=load, out=out[3]);
    Bit(in=in[4], load=load, out=out[4]);
    Bit(in=in[5], load=load, out=out[5]);
    Bit(in=in[6], load=load, out=out[6]);
    Bit(in=in[7], load=load, out=out[7]);
    Bit(in=in[8], load=load, out=out[8]);
    Bit(in=in[9], load=load, out=out[9]);
    Bit(in=in[10], load=load, out=out[10]);
    Bit(in=in[11], load=load, out=out[11]);
    Bit(in=in[12], load=load, out=out[12]);
    Bit(in=in[13], load=load, out=out[13]);
    Bit(in=in[14], load=load, out=out[14]);
    Bit(in=in[15], load=load, out=out[15]);
}
```



Simulator output: Register

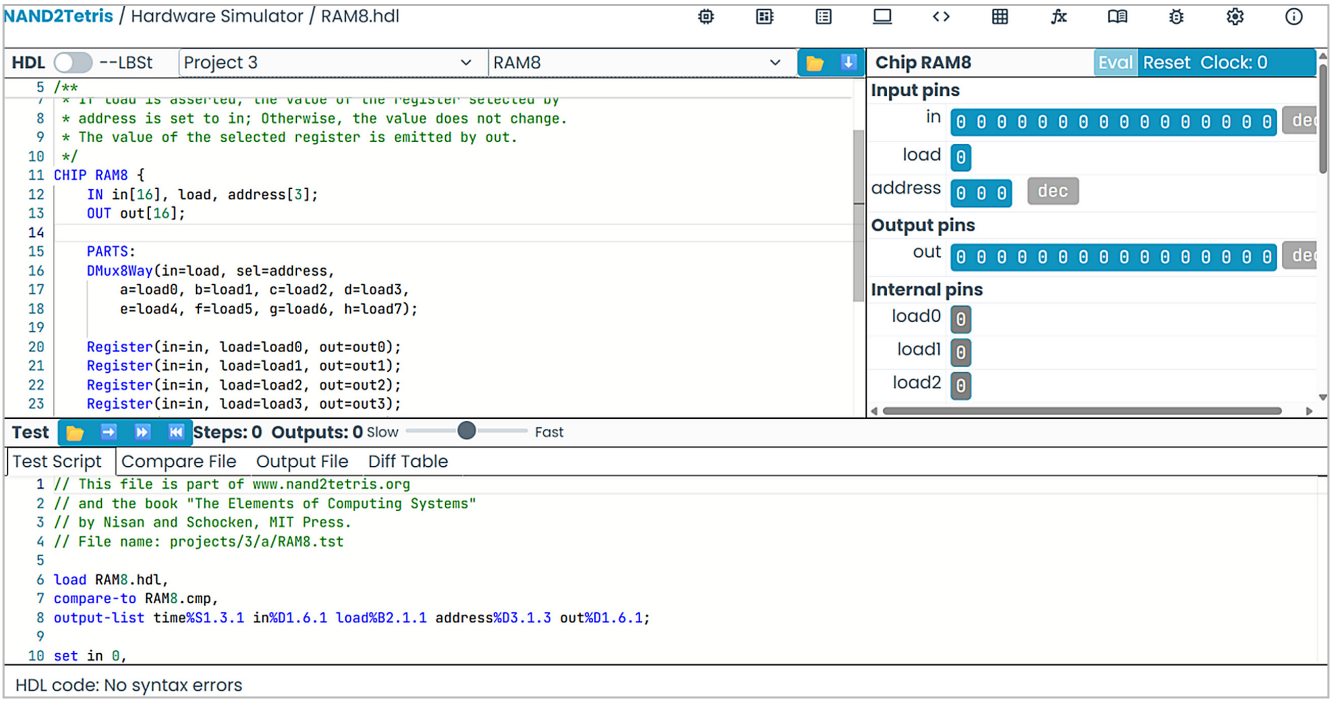
RAM8

```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address,
        a=load0, b=load1, c=load2, d=load3,
        e=load4, f=load5, g=load6, h=load7);

    Register(in=in, load=load0, out=out0);
    Register(in=in, load=load1, out=out1);
    Register(in=in, load=load2, out=out2);
    Register(in=in, load=load3, out=out3);
    Register(in=in, load=load4, out=out4);
    Register(in=in, load=load5, out=out5);
    Register(in=in, load=load6, out=out6);
    Register(in=in, load=load7, out=out7);

    Mux8Way16(
        a=out0, b=out1, c=out2, d=out3,
        e=out4, f=out5, g=out6, h=out7,
        sel=address, out=out);
}
```



Simulator output: RAM8

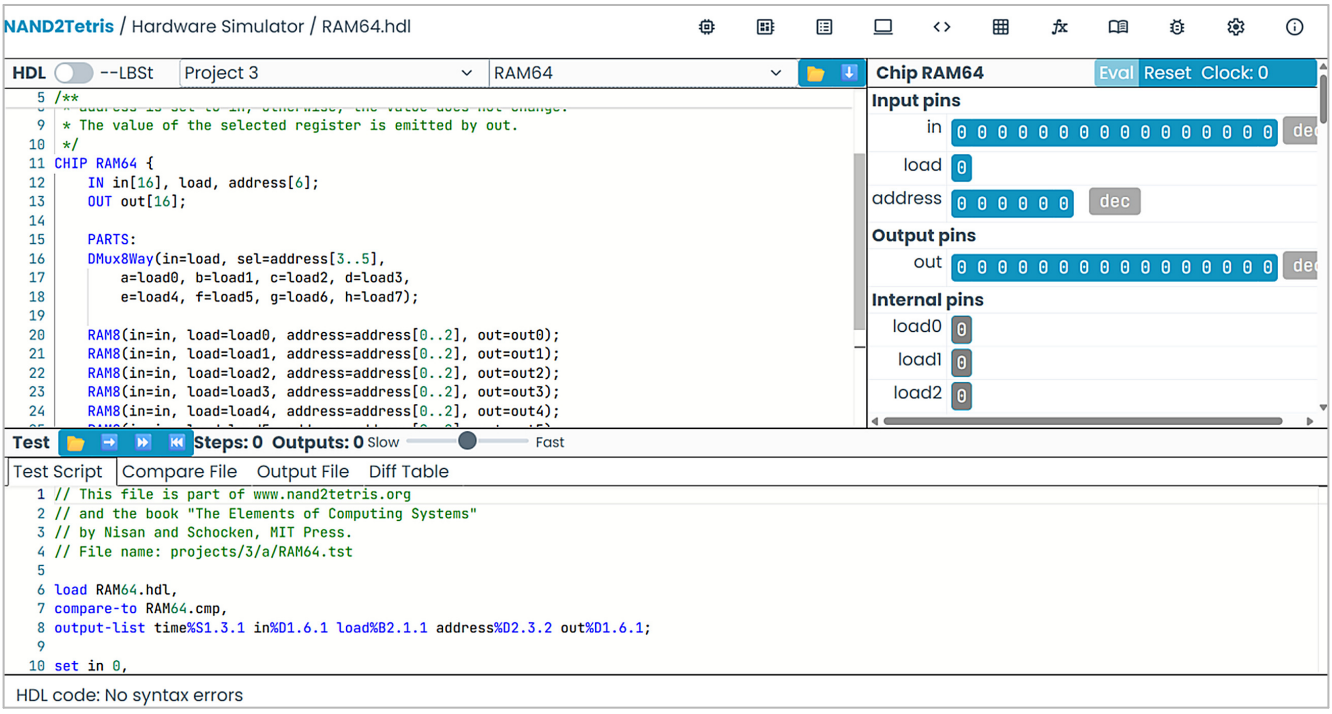
RAM64

```
CHIP RAM64 {
    IN in[16], load, address[6];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address[3..5],
        a=load0, b=load1, c=load2, d=load3,
        e=load4, f=load5, g=load6, h=load7);

    RAM8(in=in, load=load0, address=address[0..2], out=out0);
    RAM8(in=in, load=load1, address=address[0..2], out=out1);
    RAM8(in=in, load=load2, address=address[0..2], out=out2);
    RAM8(in=in, load=load3, address=address[0..2], out=out3);
    RAM8(in=in, load=load4, address=address[0..2], out=out4);
    RAM8(in=in, load=load5, address=address[0..2], out=out5);
    RAM8(in=in, load=load6, address=address[0..2], out=out6);
    RAM8(in=in, load=load7, address=address[0..2], out=out7);

    Mux8Way16(
        a=out0, b=out1, c=out2, d=out3,
        e=out4, f=out5, g=out6, h=out7,
        sel=address[3..5], out=out);
}
```



Simulator output: RAM64

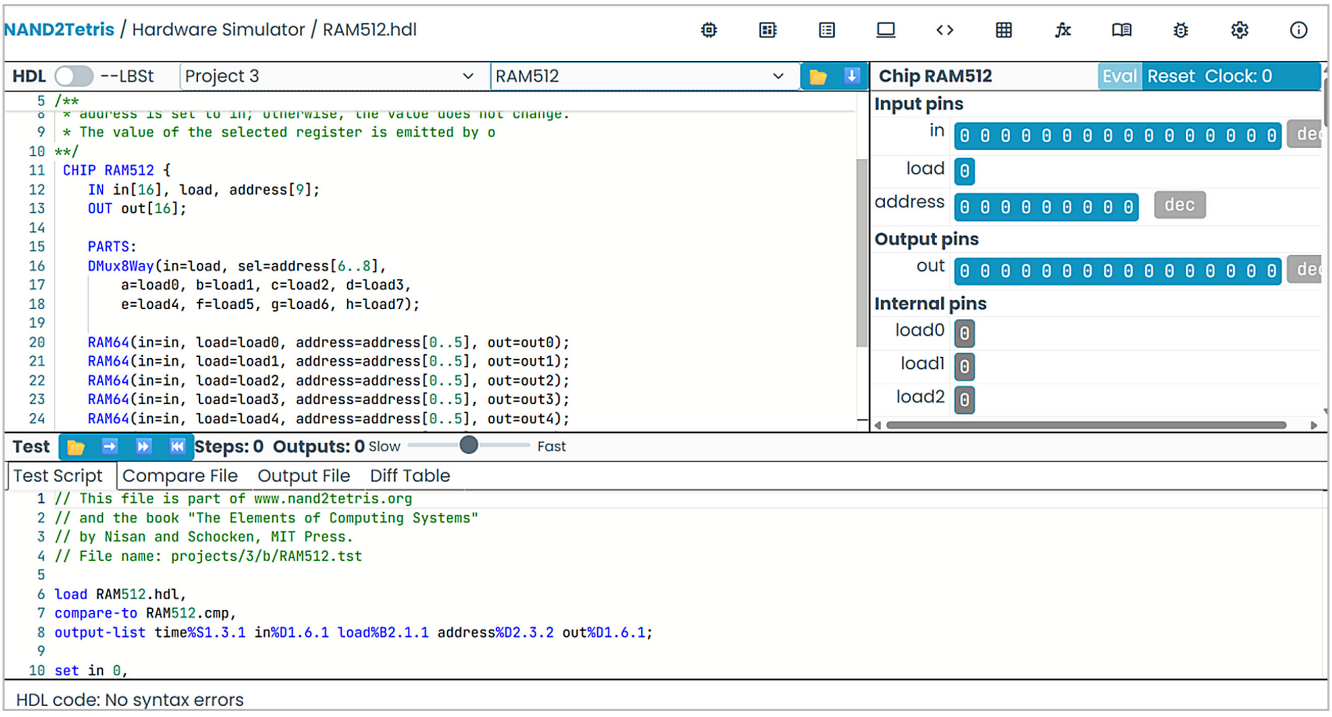
RAM512

```
CHIP RAM512 {
    IN in[16], load, address[9];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address[6..8],
        a=load0, b=load1, c=load2, d=load3,
        e=load4, f=load5, g=load6, h=load7);

    RAM64(in=in, load=load0, address=address[0..5], out=out0);
    RAM64(in=in, load=load1, address=address[0..5], out=out1);
    RAM64(in=in, load=load2, address=address[0..5], out=out2);
    RAM64(in=in, load=load3, address=address[0..5], out=out3);
    RAM64(in=in, load=load4, address=address[0..5], out=out4);
    RAM64(in=in, load=load5, address=address[0..5], out=out5);
    RAM64(in=in, load=load6, address=address[0..5], out=out6);
    RAM64(in=in, load=load7, address=address[0..5], out=out7);

    Mux8Way16(a=out0, b=out1, c=out2, d=out3,
        e=out4, f=out5, g=out6, h=out7,
        sel=address[6..8], out=out);
}
```



Simulator output: RAM512

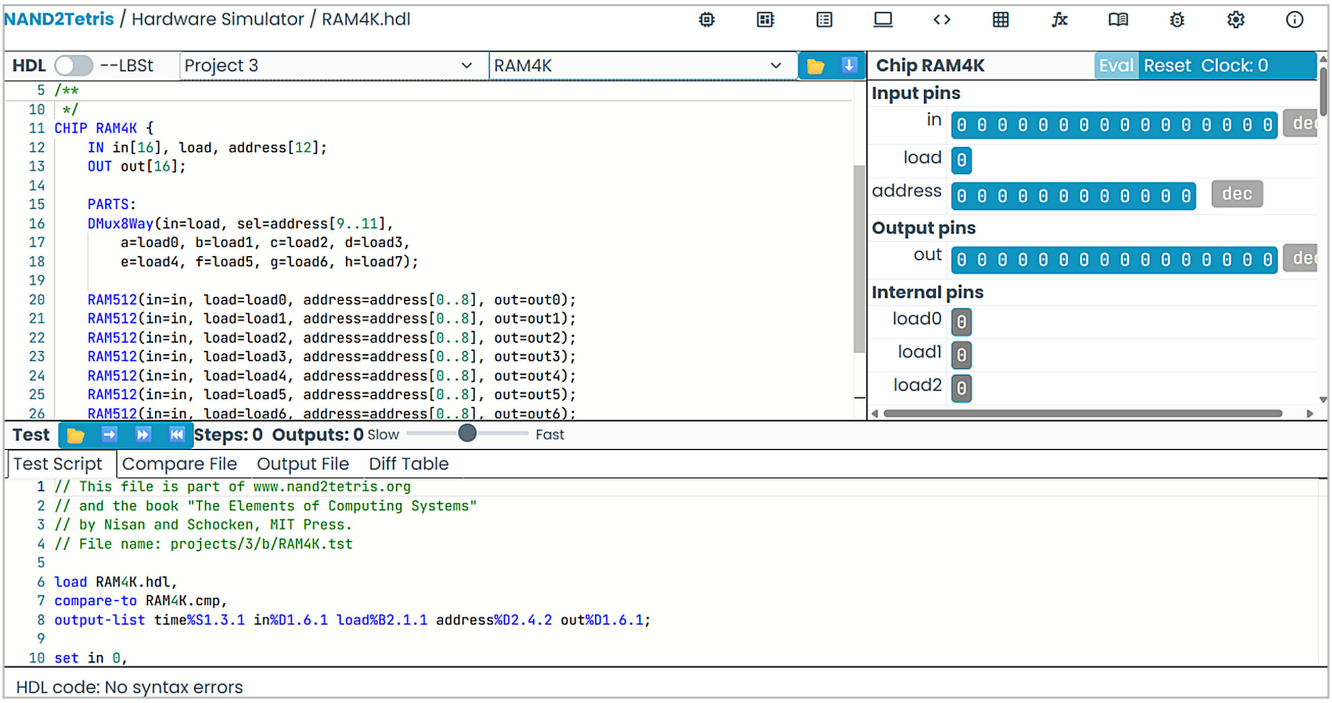
RAM4K

```
CHIP RAM4K {
    IN in[16], load, address[12];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address[9..11],
        a=load0, b=load1, c=load2, d=load3,
        e=load4, f=load5, g=load6, h=load7);

    RAM512(in=in, load=load0, address=address[0..8], out=out0);
    RAM512(in=in, load=load1, address=address[0..8], out=out1);
    RAM512(in=in, load=load2, address=address[0..8], out=out2);
    RAM512(in=in, load=load3, address=address[0..8], out=out3);
    RAM512(in=in, load=load4, address=address[0..8], out=out4);
    RAM512(in=in, load=load5, address=address[0..8], out=out5);
    RAM512(in=in, load=load6, address=address[0..8], out=out6);
    RAM512(in=in, load=load7, address=address[0..8], out=out7);

    Mux8Way16(a=out0, b=out1, c=out2, d=out3,
        e=out4, f=out5, g=out6, h=out7,
        sel=address[9..11], out=out);
}
```



Simulator output: RAM4K

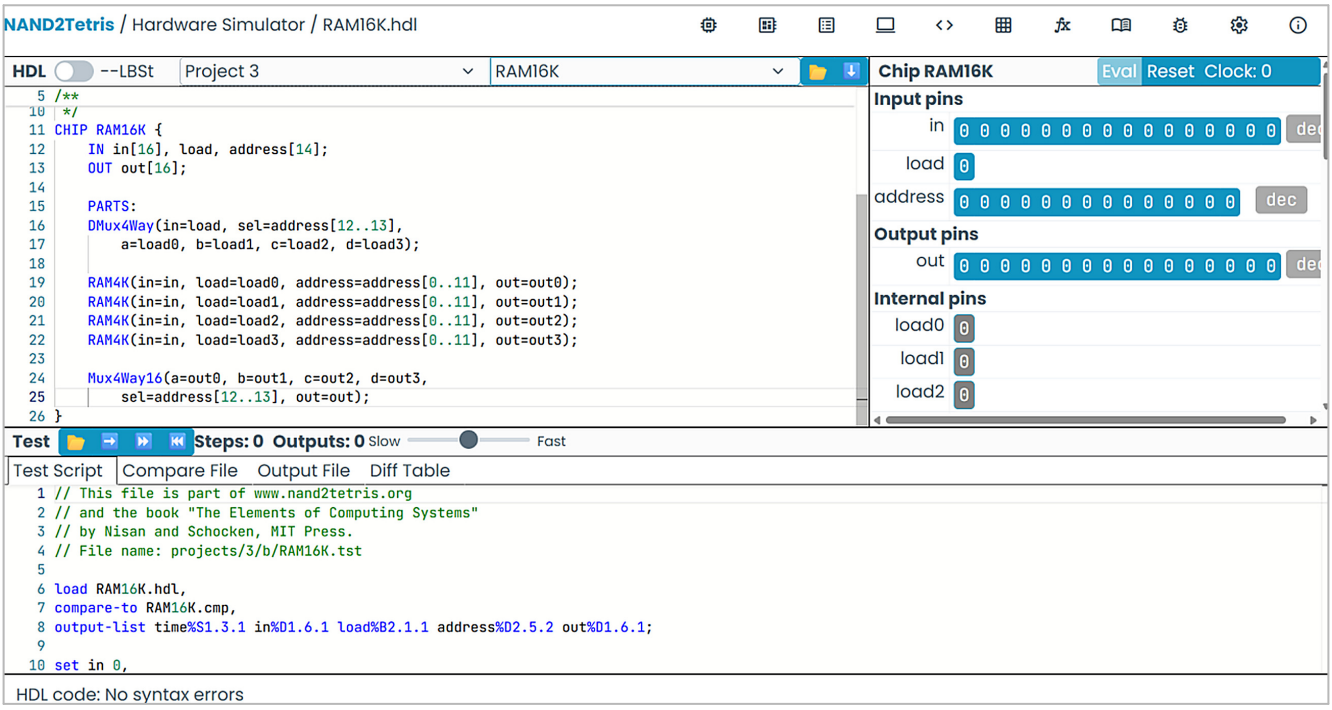
RAM16K

```
CHIP RAM16K {
    IN in[16], load, address[14];
    OUT out[16];

    PARTS:
    DMux4Way(in=load, sel=address[12..13],
        a=load0, b=load1, c=load2, d=load3);

    RAM4K(in=in, load=load0, address=address[0..11], out=out0);
    RAM4K(in=in, load=load1, address=address[0..11], out=out1);
    RAM4K(in=in, load=load2, address=address[0..11], out=out2);
    RAM4K(in=in, load=load3, address=address[0..11], out=out3);

    Mux4Way16(a=out0, b=out1, c=out2, d=out3,
        sel=address[12..13], out=out);
}
```



Simulator output: RAM16K

PC

```
CHIP PC {
    IN in[16], load, inc, reset;
    OUT out[16];

    PARTS:
    // Increment current PC value
    Inc16(in=pcOut, out=incOut);

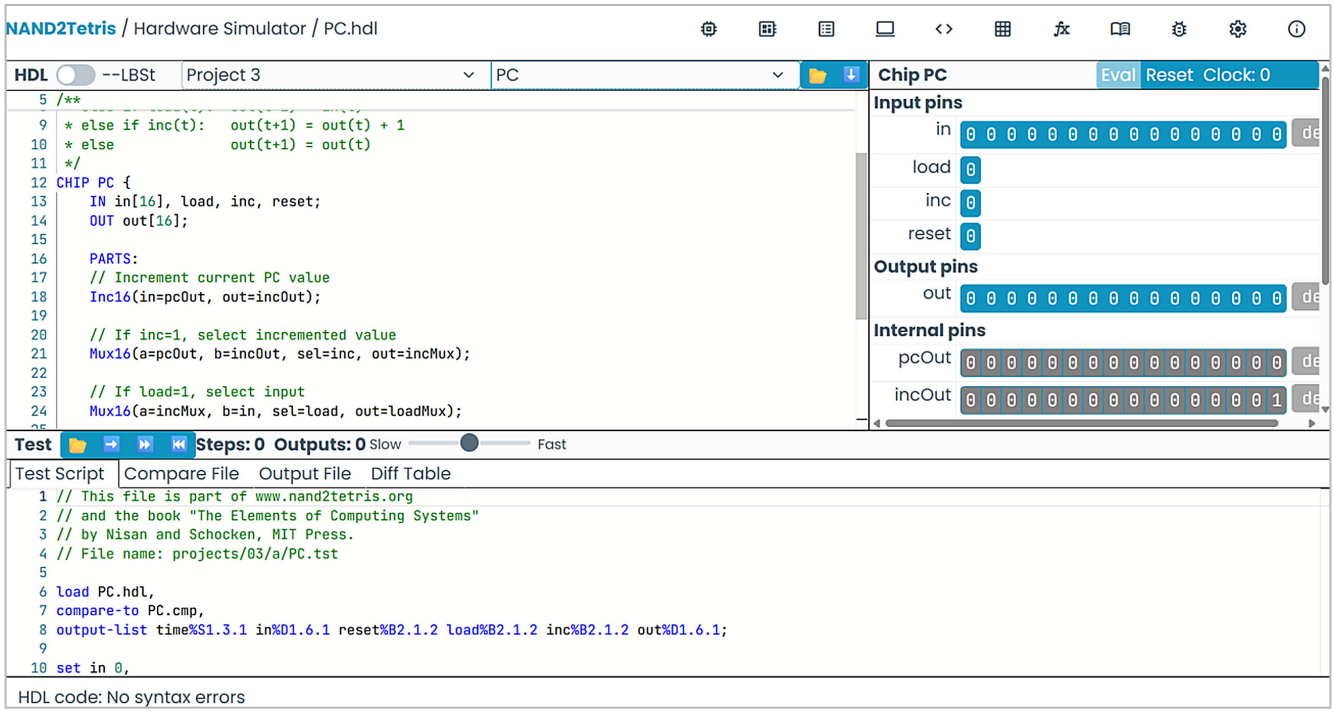
    // If inc=1, select incremented value
    Mux16(a=pcOut, b=incOut, sel=inc, out=incMux);

    // If load=1, select input
    Mux16(a=incMux, b=in, sel=load, out=loadMux);

    // If reset=1, select 0
    Mux16(a=loadMux, b=false, sel=reset, out=resetMux);

    // Store the new PC value
    Register(in=resetMux, load=true, out=pcOut);

    // Send internal PC value to output
    Mux16(a=pcOut, b=pcOut, sel=false, out=out);
}
```



Simulator output: PC